

02-510/710: Computational Genomics, Spring 2026

HW3: Machine learning and TF binding

Version: 1

Due: 23:59 EST, Mar 22, 2026 on Canvas

Topics in this assignment:

1. ChIP-seq
2. Classification
3. TF Binding and Supervised Learning

What to hand in.

- One write-up (in PDF format) addressing each of the questions.
- All source code used to generate the results (scripts and/or notebooks).

Submit the PDF write-up and the `classification.*` files separately to Canvas.

Please note that all the solutions and code must be your own. We will check for plagiarism after the final submission.

1. [30 points] ChIP-seq.

We provide you with processed ChIP-seq peak data in `trimmed.bed`. This is real transcription factor ChIP-seq data from a cancer cell line. In this assignment, you will use several common genomics tools to analyze transcription factor binding sites and interpret their biological significance. First, you will obtain the genomic sequences underlying the ChIP-seq peaks and use motif discovery tools to identify enriched DNA sequence motifs that may correspond to transcription factor binding preferences. You will then compare the discovered motifs to known transcription factor motifs. Finally, you will interpret the potential biological roles of the binding sites using GREAT (Genomic Regions Enrichment of Annotations Tool). GREAT associates genomic regions such as ChIP-seq peaks with nearby genes and tests whether particular biological processes or pathways are enriched among those genes, often using Gene Ontology (GO) annotations. This allows researchers to connect sets of regulatory regions to the biological functions they may regulate.

Using `peak.bed`, please complete the following tasks.

- Get the genomic sequence for each of the intervals through UCSC Genome Browser.
Step 1: Uploading the data. Upload your "`trimmed.bed`" file to the UCSC Genome Browser (<https://hgwl.soe.ucsc.edu/cgi-bin/hgCustom>) as a custom track. Ensure you select GRCh37/hg19 as the reference assembly.
Step 2: Accessing the Table Browser. Use the `TableBrowser` in UCSC Genome Browser. Proceed to view the first annotation listed from your uploaded custom track.
Step 3: Extracting Sequence Data. In the Table Browser, set the output format to **sequence** and name your output file **trimmed.fasta**. Initiate the output process to receive your data. Don't change other output settings.
- Use MEME to call enriched motifs based on the output sequence.
Step 1: Upload your **trimmed.fasta** file to MEME (<http://meme-suite.org/tools/meme>). Under "Advanced options" set the motif width to be from 5 to 9. Don't change the other settings. Note that you may need to adjust the size of the input file to keep the number of characters within 60,000 in the file.
Step 2: Using each of your significant hits from MEME, feed them to TOMTOM by clicking the 'submit/download' arrow to the right of your MEME results. Use all default options for TOMTOM.
- Exploring Biological Processes with GREAT.
Upload the provided **peak.bed** file to GREAT (<http://great.stanford.edu/public/html/>) to identify enriched biological processes and pathways. **Be sure to use GREAT version 3.0.0, with species hg19.** Note that **trimmed.fasta** is a subset of **peak.bed**. Use the default settings for the analysis.

Questions

- (a) What are the meanings of the three columns present in the provided `trimmed.bed` file?

Solution

- (b) Share a screenshot of the output you received from MEME and identify which motif appears to be the most promising, based on the discovered motif. Explain your criteria for selection.

Solution

- (c) After determining the most promising motif from MEME, use TomTom to find the closest match. What is the top hit obtained from TomTom for the most promising motif from MEME? Provide a Screenshot of the output returned by TomTom.

Solution

- (d) For **each** of the motifs from MEME, find the top hit using TomTom. What do the TomTom results suggest? How does JASPAR relate to this process?

Solution

- (e) Provide a screenshot of the top ten GO Biological Process hits from GREAT. How do these pathways align with your expectations from the MEME analysis? Discuss the statistical significance of these pathways, elaborating on the meanings of key enrichment statistics.

Solution

- (f) Considering your analysis in GREAT, evaluate the usefulness of the provided TF ChIP-Seq data (peak.bed) for determining the cancer type of the cells from which the provided data was collected. Provide a detailed explanation supporting your assessment.

Solution

- (g) How could single-cell sequencing technologies enhance the insights gained from the ChIP-seq data analysis provided in this assignment?

Solution

- (h) Reflect on the limitations of using ChIP-seq data for understanding transcription factor binding landscapes in cancer cells. What additional experiments or data would complement this analysis for a more comprehensive understanding? Explain.

Solution

2. [30 points] Cell type classification

While cells from the same individual share (roughly) the same DNA sequence, different cells use different subsets of these genes, and at different levels. Single-cell RNA sequencing (scRNA-seq) measures the activity levels of a set of annotated genes in a single cell, a vector with continuous values that is termed ‘gene expression profile’. The type of a cell (e.g., lung cell, brain cell, skin epidermis) is closely connected with its gene expression profile.

In this problem you will use a combination of unsupervised and supervised learning techniques to explore and classify cells from a large single-cell RNA sequencing (scRNA-seq) dataset. This is an important goal. For example, when taking a cancer biopsy the sample contains several different types of cells and the ability to accurately determine the cell composition can have important impact on the type of treatment prescribed.

Data Description We will implement several learning algorithms in Python and test them on an RNA-seq dataset. All the required data is in the `dataset/` folder. The two files `train_gene_expression.npz` and `test_gene_expression.npz` are sparse SciPy/Numpy-formatted gene expression matrices. Each row of the matrix is a cell, with its expression value for each gene in a separate column; there are a total of 22289 genes (columns) in each matrix. In addition, the two files `train_labels.npy` and `test_labels.npy` contain cell types labels for each row in the training and testing datasets, respectively; there are a total of five cell types present in both the train and test data.

Before starting: make sure you have Numpy, Matplotlib, Pandas, Seaborn, SciPy, SciKit Learn and PyTorch (the CPU version is sufficient) installed on your machine. If you have trouble installing these packages, please come to office hours for help.

Your task:

Note: Your code is graded by an autograder. You have been given a skeleton code `classification.py`. You have to complete 3 functions (`reduce_dimensionality_pca`, `plot_transformed_cells`, and `train_and_evaluate_svm_classifier`). The details about the functions are given later. At the end, the script should be able to run with command line:

```
python classification.py q2_data/train_gene_expression.npz \
q2_data/test_gene_expression.npz q2_data/train_labels.npy \
q2_data/test_labels.npy "svm_pipeline"
```

The formats of arguments and return values of each function are explicitly described in the skeleton code. For the hyperparameters, please use the **default** values specified in the skeleton code.

- (a) (10 points) Not all genes are useful for cell type classification. To save time and improve accuracy we can cluster using only a subset of the genes. One common criteria to select genes is dispersion, which is defined as the variance divided by mean. The `get_top_gene_filter` function is to compute a mask that selects only the columns of the expression matrix that are the top 2000 most dispersed across all cells in the training dataset.

As another preprocessing step, we will use Principal Component Analysis (PCA) to reduce the dimensionality of the filtered genes. PCA is an unsupervised learning technique that computes linear combinations of features that best capture the axes of variance in the high-dimensional feature space. **Complete** the function `reduce_dimensionality_pca`, which performs PCA on the filtered expression data and returns the transformed training data and test data. Please use the SciKit Learn implementation of PCA (see documentation [here](#)).

(**Hint:** for best results, concatenate the filtered training and expression data together and input the combined data for training the PCA model.)

- (b) (10 points) To visualize the appropriateness of PCA embedding for the cell classification task, we can plot our cells in the 2D plane spanned by the top 2 principal components. **Complete** the `plot_transformed_cells` function, which takes as input the transformed gene expression data and plots the cells using only the first two principal components represented in the transformed data. Your plot should color the cells by their cell type label. Apply the completed function to the PCA-transformed *training* gene expression data, and **attach an image** of your plot below. **Comment** on how clear the difference between cell types is in your plot.

(**Hint:** consider using Pandas + Seaborn for plotting and labeling.)

Solution

- (c) (10 points) Next, we will implement a pipeline that uses a Support Vector Machine (SVM) classifier to classify our cells. **Complete** the `train_and_evaluate_svm_classifier` function, which trains a simple SVM pipeline on the PCA-reduced gene expression data and returns the trained model as well as the classification accuracy on the test data. Please use the SciKit Learn implementation of `LinearSVC` (see documentation [here](#) and [here](#)).

Last, we will combine the above functions to generate a pipeline for cell-type classification. The (`mode == "svm_pipeline"`) branch within the `__main__` method of the Python script has been provided for you, which combines gene filtering, PCA dimensionality reduction, and SVM classification and prints the final score of your classifier. **Report** the training and test accuracy of your classifier below.

Solution

3. [30 points] TF Binding and Supervised Learning.

This question is a fictional story about two transcription factors (TFs), CTCF and ZNF143, that was inspired by real events.

Once upon a time, CTCF and ZNF143 were both thought to play important roles in three-dimensional DNA interactions. They were believed to bind DNA at the bases of chromatin loops and to bind near each other in ways that stabilized each other's binding. At this time, CTCF had a well-characterized motif, but the motif for ZNF143 was unknown.

Questions

- (a) Your collaborator did ChIP-seq on CTCF and ZNF143 binding in liver and adipose, doing 2 replicates in each. The CTCF ChIP-seq had NSC 1.09 for both liver replicates and 1.08 for both adipose replicates. The ZNF143 ChIP-seq had NSC 1.09 for both liver replicates, 1.08 for one adipose replicate, and 1.01 for the other adipose replicate. The CTCF ChIP-seq had 60,000 reproducible peaks in liver and 59,000 reproducible peaks in adipose, while the ZNF143 ChIP-seq had 25,000 reproducible peaks in liver and 3,000 reproducible peaks in adipose. Provide 2 reasons why the number of reproducible peaks for ZNF143 might differ between tissues.

Solution

- (b) You run MEME on the reproducible peaks in all datasets and get the known motif for CTCF. Provide 3 reasons why the motif found for ZNF143 might be the same as the motif found for CTCF.

Solution

- (c) You want to train a machine learning model to predict CTCF binding in each of the liver and adipose. 25% of the liver CTCF peaks are not found in either adipose replicate, and 20% of the adipose CTCF peaks are not found in either liver replicate. For each of these models, would you train a logistic regression or a CNN?

Solution

- (d) You train a machine learning model for predicting CTCF binding in liver and adipose as well as a machine learning model for predicting ZNF143 binding in liver and adipose. For the negative set, you use random regions of the genome that have the same G/C-content as the peaks because the CTCF motif is G/C-rich and you want to make sure that the model does not just learn G/C-content, where you make 10 negatives for every positive. The models produce the following AUROC values:

- CTCF liver: training AUROC 0.91, validation AUROC 0.90
- CTCF adipose: training AUROC 0.94, validation AUROC 0.73
- ZNF143 liver: training AUROC 0.82, validation AUROC 0.81
- ZNF143 adipose: training AUROC 0.61, validation AUROC 0.60

Why might the models have different performance levels? Why might the training and validation performances differ for some models but not others?

Solution

(e) Your machine learning models have the following AUPRCs:

- CTCF liver: training 0.81, validation 0.80
- CTCF adipose: training 0.84, validation 0.66
- ZNF143 liver: training 0.73, validation 0.72
- ZNF143 adipose: training 0.51, validation 0.51

Why might all of the AUPRCs be lower than all of the AUROCs?

Solution

(f) You are interested in predicting whether a region will be bound by only CTCF or only ZNF143 in the liver. You could use your CTCF and ZNF143 models to make predictions of CTCF and ZNF143 binding in the liver and see whether only 1 model predicts a TF binding site. Based on the results you have seen, are you confident that this is a good approach? If so, what results have made you confident? If not, what additional results would you want to see?

Solution

(g)) Your lab-mate comes to you with some exciting news. After some additional hyper-parameter tuning, the CTCF adipose model now achieves:

- training AUROC 0.91
- validation AUROC 0.90
- training AUPRC 0.81
- validation AUPRC 0.80

Based on the results you have, are you confident that you could use your CTCF liver model and this new model to predict whether a CTCF peak will be stronger in liver than in adipose? If so, what makes you confident, and how would you do this? If not, what additional results would you want to see?

Solution